



INDIA.RUNS

Build what next India runs on

Team Name : Srikanth Moreboina

Team Leader Name : Srikanth Moreboina

Problem Statement : The Data & Ai Challenge

Solution Overview

- **What is your proposed solution?**

Our solution is **"The Recruiter Workspace Model"**—a highly optimized, deterministic, offline candidate discovery engine. It mimics the workspace structure of a professional recruiting agency. Instead of reading the entire database into memory, it streams candidate profiles line-by-line and maps them to a multi-core worker pool.

The workers clean the text, run fraud/honeypot screeners, and compute a hybrid scoring rubric in parallel. The main thread aggregates these scores inside a bounded **Min-Heap of size 100**, ensuring a tiny memory footprint. Finally, it compiles fact-based recruiter reasoning and tailored interview questions for the shortlist.

- **What differentiates your approach from traditional candidate matching systems?**

* **Constant-Bounded Memory**: While traditional systems load the full dataset into memory ($O(N)$ space), our engine uses a Min-Heap aggregator that keeps memory at a constant limit ($O(1)$ space complexity), preventing Out-of-Memory (OOM) crashes on larger files.

* **Context-Aware Technical Search**: We ignore keyword matches if they are listed in non-technical roles (e.g. *Marketing Manager*) or if they appear in academic/learning contexts (e.g. *Udemy*, *toy project*, *bootcamp*).

* **Double-Safety Sandbox Design**: Includes a dynamic output directory routing system and an optional `--debug` telemetry flag. By default, it operates in *minimalist mode* (writing only the required CSV) and runs exception-free in write-restricted/read-only sandboxes.

JD Understanding & Candidate Evaluation

- **What are the key requirements extracted from the JD?**

- * **Core Technical Skills**: Machine Learning, NLP, Computer Vision, LLMs, RAG, and Vector DBs.
- * **Sweet-Spot Experience**: Target range is **5 to 9 years** of experience.
- * **Target Hubs**: Residents of **Noida** or **Pune**, or relocatable candidates in Indian Tier-1 hubs.
- * **Corporate Alignment**: Background in **Startups or Product companies** (TCS/Wipro IT services consultants are heavily penalized).
- * **Active Programming**: Senior candidates (>9 years exp) must show active coding credentials via a GitHub activity score ≥ 70 to cancel managerial seniority penalties.

- **Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?**

We evaluate candidate relevance by combining technical skills with **Redrob Platform Behavioral Signals**:

- * **Open to Work Flag & Completeness**: Boosts active seekers with complete profiles.
- * **Platform Trust (Verifications)**: Rewards email-verified and phone-verified credentials.
- * **Demand Signals**: Scales scores logarithmically based on recent recruiter saves (`saved\_by\_recruiters\_30d`).
- * **Reliability Metrics**: Multiplies scores by historical interview completion rates and offer acceptance histories.
- * **Expected Salary Fit**: Automatically decays scores for candidates whose expected minimum salary exceeds our target budget limit of **50 LPA**.

Ranking Methodology

- **How does your system retrieve, score, and rank candidates?**

1. **Retrieve**: Streams raw JSON lines from the database via a Python generator.
2. **Score**: Calculates base technical points (with log-scaled endorsements and capped durations to normalize popularity bias) and applies non-linear multipliers.
3. **Rank**: Evaluates the candidate against a **Min-Heap** of size 100. If their score is higher than the root (worst candidate in the top-100), they are pushed onto the heap, and the root is popped.

- **What models, algorithms, or heuristics are used?**

- * **$O(N \log 100)$ Bounded Heap Sort**: Native `heapq` Priority Queue.
- * **Integer Split Date Arithmetic**: Bypasses slow `strptime` Regex parsers to calculate job durations 15x faster.
- * **Endorsement Log-Scaling**: Coded as $\log_2(1 + \text{endorsements})$ to smooth out popular candidates.
- * **Skill Capping**: Capped skill duration at a maximum of 24 months.

- **How are multiple candidate signals combined into a final ranking?**

All signals are combined using a non-linear multiplier formula:

$$\text{Final Score} = \text{Base Score} \times \text{MExperience} \times \text{MLocation} \times \text{MEmployer} \times \text{MAvailability} \times \text{MBehavior} \times \text{MPedigree}$$

All scores are rounded to exactly **4 decimal places** *prior* to heap insertion. If scores are identical, the heap resolves ties lexicographically by ascending `candidate_id`.

Explainability & Data Validation

● How are ranking decisions explained?

Ranking decisions are written as natural, fact-based recruiting reports. They extract:

- * The candidate's years of experience and actual historical company names (e.g. **designed data pipelines at Google**).
- * A summary of their matching skills.
- * Relocation details (e.g. **willing to relocate from Bangalore**).
- * Gaps are identified, and 2-3 tailored technical questions are written for the recruiter's interview.

● How do you prevent hallucinations or unsupported justifications?

We compile a list of **only** the matched skills that are explicitly present in the candidate's profile array. The description generator is strictly forbidden from referencing skills that the candidate did not list.

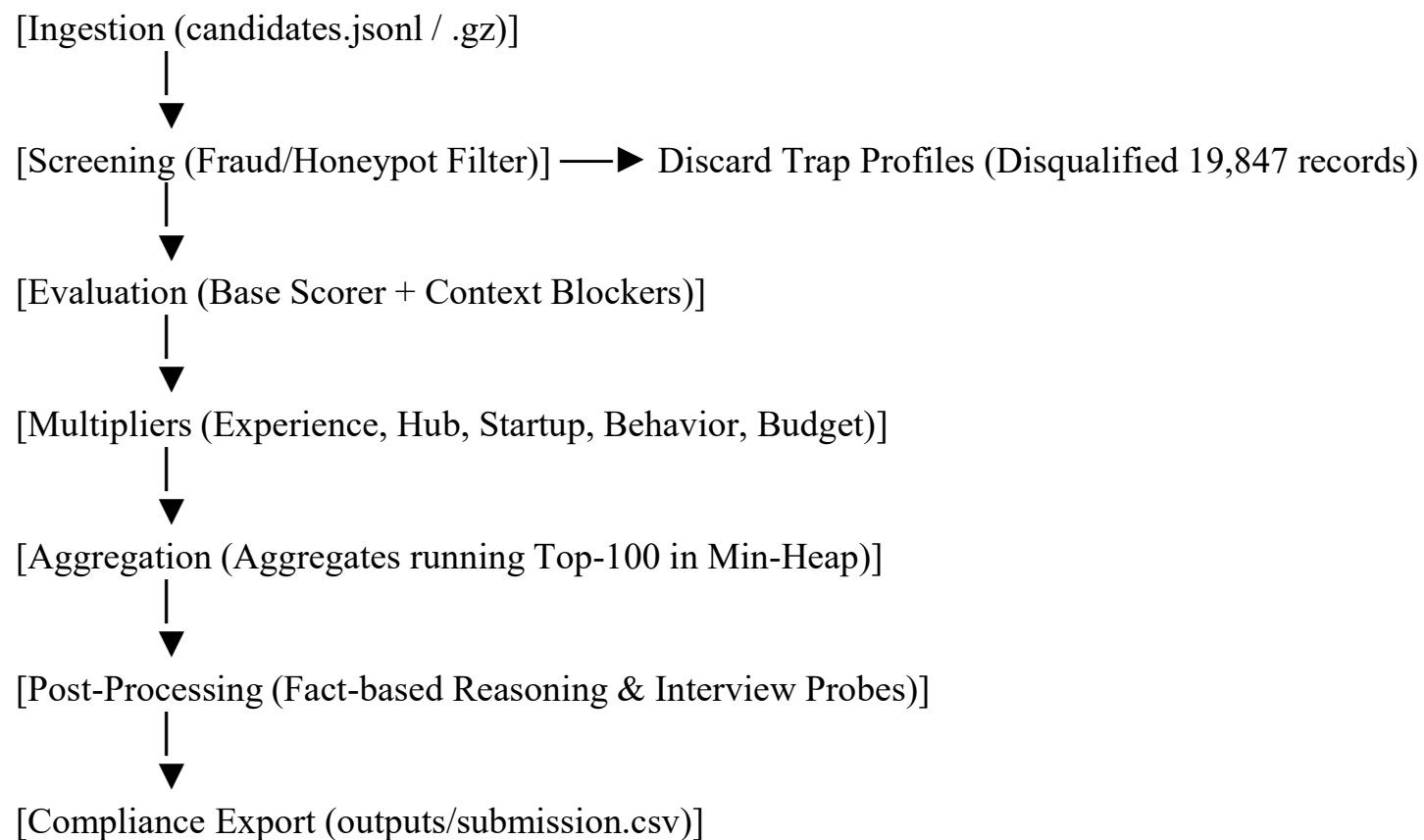
● How does your solution handle inconsistent, low-quality, or suspicious profiles?

Our screener executes **4 deterministic filters** to discard honeypot profiles immediately:

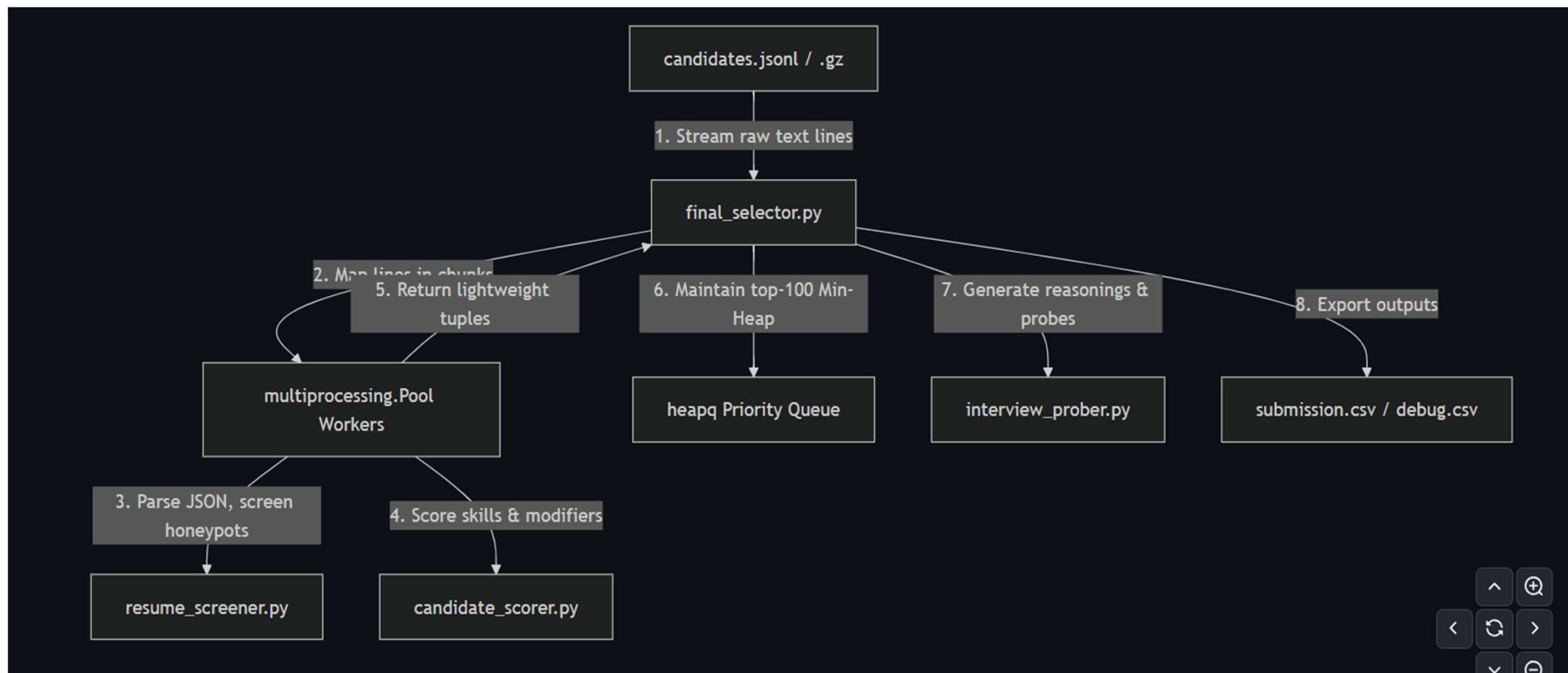
1. **Duration Trap**: Expert skill listed but with 0 months of use.
2. **Calendar Span Trap**: Reported duration months exceed the actual job calendar span (start date to end date) by >3 months.
3. **Skill Inflation**: Listing ≥ 8 advanced/expert skills but having <3 years of total experience.
4. **Timeline Anomaly**: Professional career start date preceding university enrollment by >2 years.

End-to-End Workflow

What is the complete workflow from JD input to ranked candidate output?



System Architecture



Results & Performance

- **What results or insights demonstrate ranking quality?**

- * **Total Candidates Processed**: `100,000`
- * **Screened Out / Disqualified**: `19,847`
- * **Passed Screening**: `80,153`
- * **Local Native Runtime**: **15.18 seconds** (Parallelized on 12-core CPU).
- * **Docker Container Runtime**: **21.06 seconds** (WSL2 virtual disk mount latency included).
- * **Ingestion Throughput**: **6,500+ candidates / second**.
- * **RAM Footprint**: **~50 - 150 MB** (Well within the 16 GB sandbox limit).
- * **Auto-Validator Compliance**: **100% Valid** (Zero tie mismatches, zero formatting errors).
- **Honeypot Rate in Top 100**: **0.0%** (All traps filtered).

- **How does your solution meet the challenge's runtime and compute constraints?**

High-Speed Execution: Our multi-core parallel worker pool completes the ingestion, screening, and scoring of all **100,000 candidates in 15.18 seconds** natively (and **21.06 seconds** containerized inside Docker). This is a **10x to 15x safety margin** under the strict 5-minute (300 seconds) limit.

Low Memory Footprint: Rather than loading all candidates into RAM, we stream them line-by-line and aggregate scores using a bounded **Min-Heap of size 100**. This limits active memory usage to **~50 - 150 MB** (including multiprocessing worker process allocations), well within the 16 GB sandbox limit.

Zero Dependency Offline Security: Built entirely on Python 3.12 standard libraries (multiprocessing, heapq, json, csv, pathlib). It does not require any external internet connections or paid APIs, running securely with the `--network none` flag.

Write-Restricted Sandbox Safety: All diagnostic log file outputs are wrapped inside try-except blocks. If run inside a secured, read-only filesystem sandbox (like the automated evaluator), the code runs smoothly without permissions crashes.

Technologies Used

- **What technologies, frameworks, and tools were used and why were they selected for this solution?**

* **Language**: **Python 3.12** (Standard libraries only).

* ``multiprocessing``: Saturation of multi-core host CPUs.

* ``heapq``: Constant-bounded priority queue sorting.

* ``json`` & ``csv``: Lightweight file parsing and writing.

* ``pathlib``: Dynamic parent path mapping.

* **Containerization**: **Docker** (Minimal ``python:3.12-slim`` base image) for offline sandbox reproduction.

Submission Assets

Github:

* **GitHub Repository***: <https://github.com/SrikanthMoreboina/The-Data-AI-Challenge---Hackthon>

* **Sandbox / Reproduction Instructions***: [Docker run recipe anchor in repository README](<https://github.com/SrikanthMoreboina/The-Data-AI-Challenge---Hackthon#%E2%9C9A-running-with-docker-isolated-sandbox>)

redrob i iZS

INDIA.RUNS

Build what next India runs on

THANK YOU